

NATIONAL ECONOMICS UNIVERSITY, HANOI
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE



DATABASE SYSTEMS

REPORT

Topic: Personal Finance Management System

Instructor: Tran Hung

Student: Dinh Quoc Viet 11247150

Hanoi, 2026

Contents

1	Introduction	3
1.1	Problem Statement and Motivation	3
1.2	Project Objectives	3
1.3	Scope of the Project	3
1.4	Tools and Technologies	4
2	System Analysis and Requirements	5
2.1	Functional Requirements	5
2.2	Non-Functional Requirements	5
3	Database Design and Architecture	6
3.1	Entity-Relationship (ER) Modeling	6
3.2	Relational Schema Mapping	6
3.3	Database Normalization	7
3.4	Data Dictionary	8
3.4.1	Table: Users	8
3.4.2	Table: ExpenseCategories	8
3.4.3	Table: BankAccounts	8
3.4.4	Table: Expenses	8
3.4.5	Table: BalanceHistory	9
4	Database Implementation & Advanced SQL	10
4.1	Sample Data Injection	10
4.2	Indexing Strategy	10
4.3	SQL Views for Data Abstraction	12
4.4	Stored Procedures for Transaction Safety	12
4.5	Automated Triggers for ACID Compliance	14
4.6	Backup & Recovery Protocols	14
5	Python Application Development	15
5.1	System Architecture	15
5.2	Main CLI Application	15
5.3	Financial Analytics and Visualization	16
6	System Testing and Evaluation	18
6.1	Test Case 1: Transaction Integrity (Trigger Evaluation)	18
6.2	Test Case 2: Constraint Violation (Overdraft Prevention)	18
7	Database Security and Administration	18
7.1	Comprehensive Audit Trails	18
7.2	Access Control and Privileges	18

8	Conclusion and Recommendations	19
8.1	Conclusion	19
8.2	Recommendations for Future Work	19
9	References	19

1 Introduction

1.1 Problem Statement and Motivation

In an increasingly complex economic environment, personal financial literacy and meticulous asset tracking are no longer optional, but essential for long-term stability. Traditionally, individuals have relied on manual ledger entries or basic spreadsheet software (e.g., Microsoft Excel) to monitor their income and expenditures. However, these methods exhibit significant limitations: they are highly susceptible to human error, lack programmatic data validation, suffer from scalability issues, and fail to provide automated, real-time auditing capabilities.

To resolve these systemic inefficiencies, this project proposes the development of a robust **Personal Finance Management System (PFMS)**. By leveraging the power of Relational Database Management Systems (RDBMS), specifically MySQL, coupled with a Python-based application, the system aims to automate accounting calculations, enforce strict data integrity through relational constraints, and provide comprehensive analytical insights via dynamically generated visual reports.

1.2 Project Objectives

The primary objective of this project is to architect a highly reliable, mathematically consistent database system that empowers users to oversee their personal finances effectively. Specifically, the project aims to:

- Systematically track granular spending habits across customizable categories to identify financial leakages.
- Evaluate budget limits against real-time expenditures to prevent overdrafts and overspending.
- Ensure absolute data integrity by automating balance updates and maintaining a tamper-proof audit trail.
- Demonstrate the practical application of advanced SQL objects (Views, Procedures, Functions, Triggers) in solving real-world business logic.

1.3 Scope of the Project

This project emphasizes advanced backend relational database design and Python application development. The implemented architecture delves deeply into normalized schemas (up to 3NF), strategic indexing, complex aggregated views, scalar functions, parameterized stored procedures, and fully automated triggers. While web-based platforms or mobile interfaces are recognized as valuable for future expansion, the current scope strictly focuses on delivering a high-performance, feature-rich Command-Line Interface (CLI) application that acts as a robust proof-of-concept for the underlying database logic.

1.4 Tools and Technologies

The project relies on a modern, industry-standard technology stack:

- **Database Management System:** MySQL 8.x (Chosen for its robust ACID compliance).
- **Database Administration Tool:** MySQL Workbench (Utilized for ERD modeling and script execution).
- **Programming Language:** Python 3.x (Chosen for rapid prototyping).
- **Python Libraries:** `mysql-connector-python` (For secure database connectivity), `matplotlib` (For rendering mathematical data visualizations).

2 System Analysis and Requirements

2.1 Functional Requirements

The PFMS must provide the following core functionalities to fulfill its business objectives:

- **FR1 - User Management:** The system must allow the registration of new users with unique email addresses.
- **FR2 - Account Management:** Users must be able to link multiple bank accounts, defining initial balances and tracking current funds.
- **FR3 - Transaction Logging:** The system must permit users to record daily income and expenses. Every transaction must be linked to a specific bank account.
- **FR4 - Categorization:** The system must support the classification of expenses into predefined categories (e.g., Food, Transport, Utilities) for analytical purposes.
- **FR5 - Automated Auditing:** Any financial transaction must automatically recalculate the associated bank account's balance and generate an immutable record in a history ledger.
- **FR6 - Reporting & Analytics:** The system must generate monthly financial summaries (Income vs. Expense) and calculate budget status dynamically.

2.2 Non-Functional Requirements

- **NFR1 - Data Integrity:** The database must enforce strict ACID (Atomicity, Consistency, Isolation, Durability) properties. An expense must not be recorded if the account lacks sufficient funds.
- **NFR2 - Performance:** Database queries involving aggregations for monthly reports must execute in under 2 seconds, facilitated by appropriate indexing.
- **NFR3 - Security:** Application interactions must be executed via a dedicated database role with limited privileges, avoiding the use of the `root` administrator account.

3 Database Design and Architecture

3.1 Entity-Relationship (ER) Modeling

The conceptual design of the database is the foundation of the system's logic, revolving around six core entities: **Users**, **ExpenseCategories**, **BankAccounts**, **Income**, **Expenses**, and **BalanceHistory**. The ER model enforces strict business rules: A single user can hold multiple bank accounts and record numerous transactions, establishing 1-to-N relationships. Each expense must be mapped to exactly one specific category. Furthermore, each bank account is permanently tied to a historical ledger that sequentially logs every balance adjustment.

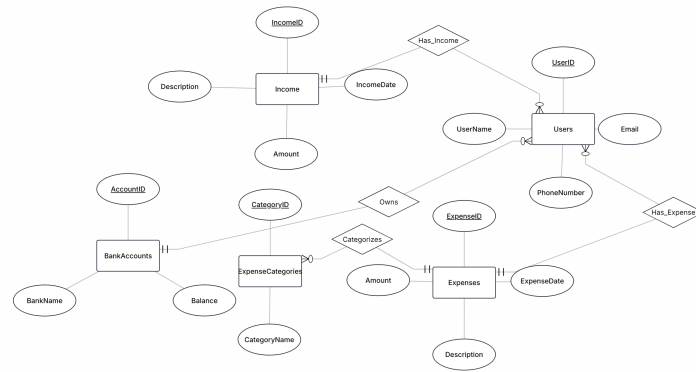


Figure 1: Entity-Relationship Diagram illustrating the core entities and their cardinalities

3.2 Relational Schema Mapping

Translating the conceptual ER model into a physical database, the relational schemas are defined as follows:

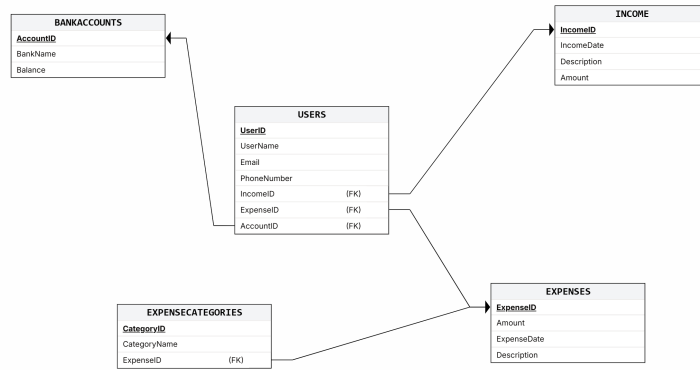


Figure 2: Relational Schema of the Database mapped from the ERD

3.3 Database Normalization

To prevent update anomalies and minimize data redundancy, the database schema was rigorously normalized up to the Third Normal Form (3NF):

- **First Normal Form (1NF):** All tables possess a primary key. Every attribute holds atomic values, and there are no repeating groups.
- **Second Normal Form (2NF):** The schema is in 1NF, and all non-key attributes are fully functionally dependent on the entire primary key. No partial dependencies exist since all primary keys in this system are single-column surrogate keys (e.g., `UserID`, `ExpenseID`).
- **Third Normal Form (3NF):** The schema is in 2NF, and no transitive dependencies exist. For instance, in the `Expenses` table, the `CategoryName` is not stored alongside the transaction; instead, only the `CategoryID` (Foreign Key) is stored, referencing the `ExpenseCategories` table. This ensures that a category name update only happens in one place.

3.4 Data Dictionary

The following tables detail the physical structure, data types, and constraints of every column within the database.

3.4.1 Table: Users

Stores the profile information of individuals using the system.

Column Name	Data Type	Key/Constraint	Description
UserID	INT	PK, AUTO_INC	Unique identifier for the user.
UserName	VARCHAR(100)	NOT NULL	Full name of the user.
Email	VARCHAR(100)	UNIQUE, NOT NULL	User's email address (must be unique).
PhoneNumber	VARCHAR(15)	NULL	User's contact number.

3.4.2 Table: ExpenseCategories

Stores predefined categories for classifying expenditures.

Column Name	Data Type	Key/Constraint	Description
CategoryID	INT	PK, AUTO_INC	Unique identifier for the category.
CategoryName	VARCHAR(50)	UNIQUE, NOT NULL	Name of the category (e.g., Food, Travel).

3.4.3 Table: BankAccounts

Stores the financial repositories linked to users.

Column Name	Data Type	Key/Constraint	Description
AccountID	INT	PK, AUTO_INC	Unique identifier for the account.
UserID	INT	FK	References Users(UserID). ON DELETE CASCADE.
BankName	VARCHAR(100)	NOT NULL	Name of the banking institution.
Balance	DECIMAL(15,2)	DEFAULT 0.00	Current available funds. Must be ≥ 0 .

3.4.4 Table: Expenses

Logs the outflow of funds.

Column Name	Data Type	Key/Constraint	Description
ExpenseID	INT	PK, AUTO_INC	Unique identifier for the expense record.
UserID	INT	FK	References Users(UserID).
CategoryID	INT	FK	References ExpenseCategories(CategoryID).
AccountID	INT	FK	References BankAccounts(AccountID).
Amount	DECIMAL(15,2)	NOT NULL	The monetary value spent. Must be > 0.
ExpenseDate	DATE	NOT NULL	The date the transaction occurred.
Description	VARCHAR(255)	NULL	Optional note regarding the expense.

3.4.5 Table: BalanceHistory

The automated ledger that records every mathematical fluctuation in account balances.

Column Name	Data Type	Key/Constraint	Description
HistoryID	INT	PK, AUTO_INC	Unique identifier for the log entry.
AccountID	INT	FK	References BankAccounts(AccountID).
ChangeType	VARCHAR(20)	NOT NULL	Specifies if it was a 'CREDIT' or 'DEBIT'.
AmountChanged	DECIMAL(15,2)	NOT NULL	The delta value of the transaction.
BalanceBefore	DECIMAL(15,2)	NOT NULL	Account balance prior to the transaction.
BalanceAfter	DECIMAL(15,2)	NOT NULL	Account balance after the transaction.
ChangedAt	DATETIME	DEFAULT NOW()	Timestamp of the operation.

4 Database Implementation & Advanced SQL

4.1 Sample Data Injection

To rigorously validate system functionalities, trigger logic, and facilitate comprehensive report generation, mock data was systematically injected into the database.

	AccountID	UserID	BankName	Balance
▶	1	1	Vietcombank	12500000.00
	2	2	Techcombank	8400000.00
	3	3	MB Bank	25000000.00
	4	4	BIDV	5600000.00
	5	5	TPBank	18900000.00
	6	6	VietinBank	9200000.00
✱	NULL	NULL	NULL	NULL

Figure 3: Sample data injected into the BankAccounts table representing initial states

	ExpenseID	UserID	CategoryID	AccountID	Amount	ExpenseDate	Description
▶	1	1	1	1	500000.00	2026-04-05	Dinner with friends
	2	1	2	1	300000.00	2026-04-10	Gasoline
	3	1	3	1	2500000.00	2026-04-15	English Course
	4	1	4	1	400000.00	2026-04-20	Cinema tickets
	5	2	1	2	600000.00	2026-04-06	Groceries
	6	3	3	3	2000000.00	2026-04-12	Books
	7	4	4	4	150000.00	2026-04-16	Netflix
	8	5	5	5	1000000.00	2026-04-20	Electricity bill
	9	6	1	6	450000.00	2026-04-22	Cafe
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 4: Sample data injected into the Expenses table for testing aggregations

4.2 Indexing Strategy

As the database scales, full table scans for financial reporting become computationally expensive. To optimize query response times, particularly for the `GROUP BY` and `WHERE` clauses used in generating financial reports, non-clustered B-Tree indexes were established.

Comment	Index_comment	Visible	Expression
		YES	NULL
		YES	NULL
		YES	NULL
		YES	NULL

Figure 5: Index definitions implemented to optimize query execution plans

4.3 SQL Views for Data Abstraction

To adhere to the principle of separation of concerns, SQL Views were constructed. These views abstract complex JOIN operations and aggregations, providing a simplified virtual table for the Python application.

The following SQL snippet illustrates the creation of the `MonthlySummaryView`, which aggregates both income and expenses using subqueries to prevent Cartesian product errors:

```
1 CREATE OR REPLACE VIEW MonthlySummaryView AS
2 SELECT
3     u.UserID, u.UserName,
4     COALESCE(i.Year, e.Year) AS SummaryYear,
5     COALESCE(i.Month, e.Month) AS SummaryMonth,
6     IFNULL(i.TotalIncome, 0) AS TotalIncome,
7     IFNULL(e.TotalExpense, 0) AS TotalExpense,
8     (IFNULL(i.TotalIncome, 0) - IFNULL(e.TotalExpense, 0)) AS NetAmount
9 FROM Users u
10 LEFT JOIN (
11     SELECT UserID, YEAR(IncomeDate) AS Year, MONTH(IncomeDate) AS Month
12     , SUM(Amount) AS TotalIncome
13     FROM Income GROUP BY UserID, YEAR(IncomeDate), MONTH(IncomeDate)
14 ) i ON u.UserID = i.UserID
15 LEFT JOIN (
16     SELECT UserID, YEAR(ExpenseDate) AS Year, MONTH(ExpenseDate) AS
17     Month, SUM(Amount) AS TotalExpense
18     FROM Expenses GROUP BY UserID, YEAR(ExpenseDate), MONTH(ExpenseDate)
19 ) e ON u.UserID = e.UserID AND i.Year = e.Year AND i.Month = e.Month;
```

Listing 1: Creation of `MonthlySummaryView`

4.4 Stored Procedures for Transaction Safety

To standardize data entry and prevent SQL injection vulnerabilities from the application layer, the business logic for all transaction insertions was strictly encapsulated into Stored Procedures.

```
1 DELIMITER $$
2 CREATE PROCEDURE AddExpenseProc(
3     IN p_UserID INT,
4     IN p_CategoryID INT,
5     IN p_AccountID INT,
6     IN p_Amount DECIMAL(15,2),
7     IN p_ExpenseDate DATE,
8     IN p_Description VARCHAR(255)
9 )
10 BEGIN
11     DECLARE v_CurrentBalance DECIMAL(15,2);
12
```

```

13  -- Check current balance
14  SELECT Balance INTO v_CurrentBalance FROM BankAccounts WHERE
AccountID = p_AccountID;
15
16  -- Validate sufficient funds
17  IF v_CurrentBalance < p_Amount THEN
18      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient funds
in the selected account.';
19  ELSE
20      INSERT INTO Expenses (UserID, CategoryID, AccountID, Amount,
ExpenseDate, Description)
21      VALUES (p_UserID, p_CategoryID, p_AccountID, p_Amount,
p_ExpenseDate, p_Description);
22  END IF;
23 END$$
24 DELIMITER ;

```

Listing 2: Stored Procedure for Adding an Expense

	IncomeID	UserID	AccountID	Amount	IncomeDate	Description
▶	7	1	1	800000.00	2026-04-25	Test Procedure
	6	6	6	10000000.00	2026-04-05	Salary April
	5	5	5	20000000.00	2026-04-02	Salary April
	4	4	4	8000000.00	2026-04-15	Freelance
	3	3	3	5000000.00	2026-04-10	Project Bonus
	2	2	2	12000000.00	2026-04-05	Salary April
	1	1	1	15000000.00	2026-04-01	Salary April
*	NULL	NULL	NULL	NULL	NULL	NULL

Figure 6: Execution of the AddIncomeProc Stored Procedure

4.5 Automated Triggers for ACID Compliance

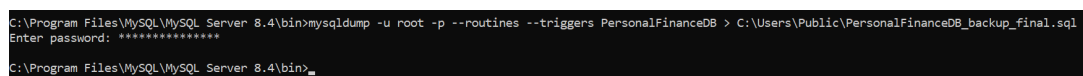
Triggers act as the immutable, automated core of the system's accounting logic. Below is the implementation of the Trigger that executes *after* a new expense is inserted. It automatically deducts the funds from the bank account and records the transaction in the history ledger.

```
1 DELIMITER $$
2 CREATE TRIGGER trg_AfterExpenseInsert
3 AFTER INSERT ON Expenses
4 FOR EACH ROW
5 BEGIN
6     DECLARE v_BalanceBefore DECIMAL(15,2);
7     DECLARE v_BalanceAfter DECIMAL(15,2);
8
9     -- Get balance before deduction
10    SELECT Balance INTO v_BalanceBefore
11    FROM BankAccounts WHERE AccountID = NEW.AccountID;
12
13    -- Calculate new balance
14    SET v_BalanceAfter = v_BalanceBefore - NEW.Amount;
15
16    -- Update BankAccounts
17    UPDATE BankAccounts
18    SET Balance = v_BalanceAfter
19    WHERE AccountID = NEW.AccountID;
20
21    -- Insert audit log into BalanceHistory
22    INSERT INTO BalanceHistory (AccountID, ChangeType, AmountChanged,
23    BalanceBefore, BalanceAfter, ReferenceType, ReferenceID, Description
24    )
25    VALUES (NEW.AccountID, 'DEBIT', NEW.Amount, v_BalanceBefore,
26    v_BalanceAfter, 'EXPENSE', NEW.ExpenseID, NEW.Description);
27 END$$
28 DELIMITER ;
```

Listing 3: Trigger for After Expense Insertion

4.6 Backup & Recovery Protocols

To safeguard against catastrophic data loss, a complete backup script utilizing the MySQL `mysqldump` utility was implemented. Crucially, the inclusion of the `--routines` and `--triggers` flags ensured that all advanced database objects were successfully archived.



```
C:\Program Files\MySQL\MySQL Server 8.4\bin>mysqldump -u root -p --routines --triggers PersonalFinanceDB > C:\Users\Public\PersonalFinanceDB_backup_final.sql
Enter password: *****
C:\Program Files\MySQL\MySQL Server 8.4\bin>
```

Figure 7: Database backup created successfully using the `mysqldump` utility

5 Python Application Development

5.1 System Architecture

The Python application acts as the interface layer for the database, structured using a modular approach. The `db.py` module handles the secure connection pool, while `main.py` manages the CLI event loop, user inputs, and calls to the respective MySQL procedures.

```
1 import mysql.connector
2
3 def get_db_connection():
4     try:
5         connection = mysql.connector.connect(
6             host="localhost",
7             user="finance_admin",    # Secure Role
8             password="Pfm@2026",    # Strict Access
9             database="PersonalFinanceDB"
10        )
11        return connection
12    except mysql.connector.Error as err:
13        print(f"Error: {err}")
14        return None
```

Listing 4: Secure Database Connection using mysql-connector

5.2 Main CLI Application

The user interaction layer provides a robust, loop-driven menu offering 18 distinct functionalities. Users can easily navigate through numerical options to register new accounts, securely log financial transactions, and review chronological history logs.


```
===== PERSONAL FINANCE MANAGEMENT SYSTEM =====
1. Add Income
2. Add Expense
3. View Monthly Summary
4. View Category-wise Spending
5. Check Budget Status
6. View Users
7. View Categories
8. View Bank Accounts
9. View Income History
10. View Expense History
11. Report: Income vs Expense Chart
12. Report: Category Spending Pie Chart
13. Add User
14. Add Bank Account
15. View Accounts by User
16. View Balance History
17. View Current Balance
0. Exit
Enter your choice: _
```

Figure 8: The comprehensive Command-Line Interface (CLI) menu of the system

5.3 Financial Analytics and Visualization

By integrating the `matplotlib` library, the Python application translates mathematical SQL outputs into visual financial analytics, aiding in rapid decision-making:

- **Bar Chart (Option 11):** Contrasts the aggregated Total Income directly against Total Expenses for a user-specified year.
- **Pie Chart (Option 12):** Breaks down monthly spending proportionally by category, highlighting where capital is being allocated.

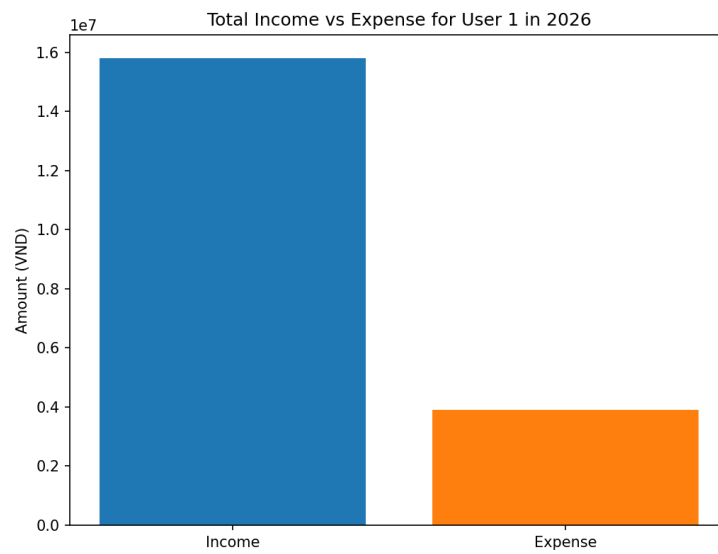


Figure 9: Income versus Expense Bar Chart rendered via matplotlib

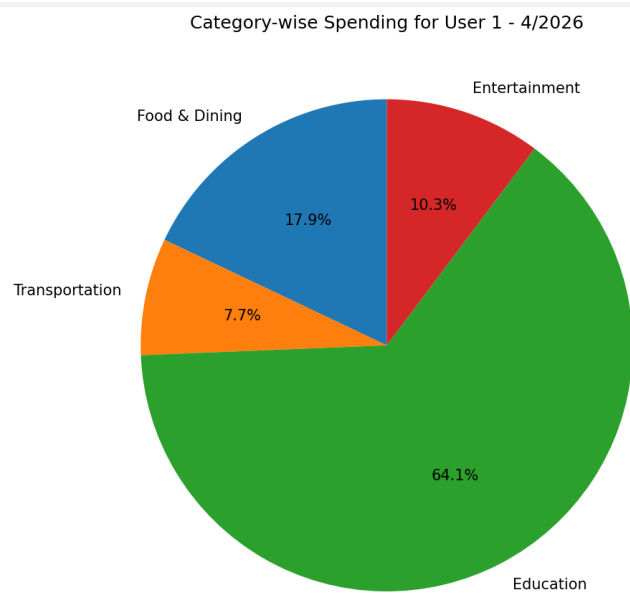


Figure 10: Category-wise Spending Pie Chart illustrating proportional expenditure

6 System Testing and Evaluation

To guarantee the reliability of the implemented business logic (Procedures, Triggers, and Constraints), a series of test cases were executed.

6.1 Test Case 1: Transaction Integrity (Trigger Evaluation)

Objective: Verify that inserting an income correctly updates the bank balance and records the history.

Action: User ID 1 inputs an income of 5,000,000 VND into Account ID 1 (Initial Balance: 12,500,000).

Expected Result: Account ID 1 balance updates to 17,500,000. `BalanceHistory` records a 'CREDIT' of 5,000,000.

Actual Result: Match. The `trg_AfterIncomeInsert` executed flawlessly, maintaining state consistency.

6.2 Test Case 2: Constraint Violation (Overdraft Prevention)

Objective: Ensure the system rejects expenses that exceed available funds.

Action: User ID 2 attempts to add an expense of 100,000,000 VND from Account ID 2 (Current Balance: 8,400,000).

Expected Result: The application throws an error; the database rejects the insertion.

Actual Result: Match. The `AddExpenseProc` procedure correctly raised the `SQLSTATE '45000'` error ("Insufficient funds"), and the transaction was rolled back.

7 Database Security and Administration

7.1 Comprehensive Audit Trails

The design of the `BalanceHistory` table represents a critical security triumph. Driven entirely by backend triggers, it guarantees a 100% transparent, tamper-proof historical record of exactly where, when, and how funds were moved, simulating a professional banking ledger.

7.2 Access Control and Privileges

Application-level security is strictly enforced by abandoning the highly vulnerable `root` user. A specialized account, `finance.admin`, was established via Data Control Language (DCL). This account is granted specific CRUD privileges only on the `PersonalFinanceDB` schema, drastically minimizing potential security vulnerabilities in the event of an application compromise.

8 Conclusion and Recommendations

8.1 Conclusion

The Personal Finance Management System successfully and comprehensively demonstrates the powerful integration of advanced relational database architecture with a functional Python application. By thoroughly leveraging MySQL’s complex procedural features—specifically Views, Stored Procedures, Functions, and automated Triggers—the system successfully automates tedious accounting calculations, ensures mathematically strict data integrity, and maintains a flawless audit trail. The Python CLI conclusively proves that the underlying database architecture is highly functional, successfully enabling users to log complex transactions and render insightful visual financial reports efficiently.

8.2 Recommendations for Future Work

While the current architecture provides a remarkably stable operational environment, future iterations could include:

- **UI/UX Modernization:** Transitioning to a Web Application using frameworks like Flask or Django to provide an intuitive Graphical User Interface.
- **Machine Learning Integration:** Implementing predictive analytics utilizing Python’s `scikit-learn` to forecast upcoming monthly expenses based on historical trends.
- **Data Exportability:** Integrating features to export complex financial reports into portable formats such as PDF or Excel files.

9 References

- MySQL 8.4 Reference Manual. Oracle Corporation.
- Python 3 Documentation. Python Software Foundation.
- `mysql-connector-python` Official Documentation. Oracle Corporation.
- `matplotlib` User Guide. The Matplotlib Development Team.
- Dinh Quoc Viet (2026). *Personal Finance Management System Source Code*. GitHub Repository: <https://github.com/vietdinh2209/personal-finance-management-system>